



Application Note: AN1200.104

LR2021 Modem Interface

Table Of Contents

1	Introduction.....	5
1.1	Purpose.....	5
1.2	Scope	5
2	SPI Communication Basics	6
2.1	SPI Write Commands	6
2.2	SPI Read Commands	7
2.3	FIFO Access	8
2.3.1	Direct Write Command.....	8
2.3.2	Direct Read Command.....	8
2.4	Monitoring FIFO Level.....	8
2.4.1	IRQ-Based FIFO Monitoring.....	8
2.4.2	Status Flag-Based FIFO Monitoring.....	8
2.4.3	API-Based FIFO Level Monitoring	9
2.4.4	Practical Implementation Strategy	9
3	General Configuration Steps	10
4	LoRa® Packet Transmission and Reception	11
4.1	LoRa Specific Configurations	11
4.2	Complete Flow for a LoRa Packet Transmission.....	11
4.3	LoRa® Packet Reception.....	12
5	FSK Packet Transmission and Reception.....	13
5.1	FSK Specific Configurations	13
5.2	FSK Packet Transmission	13
5.3	FSK Packet Reception.....	14
6	Sending/Receiving Long GFSK Packets.....	15
7	FLRC Packet Transmission and Reception	17
7.1	FLRC Specific Configurations	17
7.2	FLRC Packet Transmission	17
7.3	Required Commands Sequence for FLRC Reception	18
8	OOK Packet Transmission and Reception	19
8.1	OOK Specific Configurations.....	19
8.2	OOK Packet Transmission.....	19
8.3	OOK Packet Reception	20
9	BPSK Packet Transmission	21
9.1	BPSK Specific Configurations	21
9.2	Sequence for BPSK Packet Transmission	21
10	LR-FHSS Packet Transmission.....	23
10.1	LR-FHSS Specific Configurations	23

10.2	LR-FHSS Packet Transmission	23
11	DIO RX/TX Trigger.....	24
12	Additional Protocols	25
12.1	OQPSK_15_4 Packets.....	25
12.2	BLE Packets	25
12.3	WM-BUS Packets.....	25
12.4	WI-SUN Packets.....	26
12.5	Z-Wave Packets	26
13	Conclusion	26
14	Glossary.....	28
15	Revision History	30

Table of Figures:

Figure 1: Flowchart for transmitting payloads larger than 256 bytes. 15

Figure 2: Flowchart for receiving pay larger than 256 bytes 16

1 Introduction

The LR2021 is an advanced ultra-low power transceiver designed to operate across both Sub-GHz and 2.4GHz frequency bands, supporting a variety of modulation schemes including LoRa®, (G)FSK, 4-FSK, FLRC, BPSK, OOK, OQPSK and LR-FHSS. This versatility allows LR2021 to support a variety of wireless protocols, running in host Microcontroller, in addition to LoRaWAN: Bluetooth® Low Energy, WI-SUN, WM-BUS, 802.15.4 and Z-Wave.

First-generation LoRa transceivers relied on direct register access. Similar to LoRa generation 2 and generation 3 devices (SX1261/62/LLCC68 and LR11xx series), the LR2021 is controlled through a comprehensive API (Application Programming Interface) based on SPI commands that simplify integration and provide a more intuitive programming model.

1.1 Purpose

This application note provides detailed instructions on how to use the LR2021 Commands for transmitting and receiving different packet types with their corresponding modulations:

- Detailed implementation instructions for transmitting and receiving different packet types with their corresponding modulations
- Step-by-step command sequences with parameter explanations
- Protocol-specific configuration requirements
- Performance optimization techniques
- Common implementation pitfalls and solutions

1.2 Scope

This application note should be used in conjunction with the LR2021 datasheet.

2 SPI Communication Basics

Before diving into specific modulation types, let's understand the general command sequence for transmitting and receiving packets using the LR2021. Refer to “Host Interface” section of the LR2021 datasheet for more details.

The LR2021 is controlled via a Serial Peripheral Interface (SPI) interface operating in slave mode with a maximum frequency of 16MHz. Communication with the device follows these principles:

- The BUSY signal is used to indicate when the device is ready for commands
- Each command starts with a 16-bit opcode followed by required arguments
- The device returns status information (Stat) and IRQ status with each response

2.1 SPI Write Commands

SPI write commands are fundamental operations for controlling or configuring the LR2021 transceiver.

To ensure valid operations, adhere to the following procedures:

- Wait for the BUSY signal to be low before sending a command
- After sending a command, read the status register (Stat) from the first bytes of the response to ensure the previous command was processed successfully
- Use IRQs for event notifications

2.2 SPI Read Commands

In the LR2021, an SPI read command is a sequence where the host controller requests data from the LR2021 transceiver through the SPI bus. Unlike write commands which only send data to the device, read commands involve three steps: first sending a request, waiting, and then receiving the response data.

As detailed in Section 5.4.1.2 of the datasheet, a read command follows this sequence:

- First SPI Frame (Request):
 1. The host sends the 16-bit opcode (command identifier) and any required arguments.
 2. The LR2021 asserts the BUSY pin to indicate it's processing the command.
 3. The LR2021 returns status information (Stat) and IRQ status on the MISO line.
- Wait Period:
 1. The host waits for BUSY to go low, indicating the data is ready.
- Second SPI Frame (Response):
 1. The host initiates another SPI transaction by pulling NSS low.
 2. The host sends zeros or dummy bytes on MOSI.
 3. The LR2021 returns status information again, followed by the requested data.

The LR2021's read commands allow you to retrieve various types of information:

- Device Status Information:
 - Current chip operating mode (`GetStatus(...)`),
 - Firmware version information (`GetVersion(...)`),
 - Error flags and conditions (`GetErrors(...)`)
- Measurements and Sensor Data:
 - Battery voltage measurement (`GetVbat(...)`),
 - Temperature measurement (`GetTemp(...)`),
 - Instantaneous received signal strength (`GetRssiInst(...)`),
 - Timing information for events (`GetTimestampValue(...)`)
- FIFO and Packet Data:
 - Length of the last received packet (`GetRxPktLength(...)`),
 - Current fill level of the RX FIFO (`GetRxFifoLevel(...)`),
 - Current fill level of the TX FIFO (`GetTxFifoLevel(...)`)

Note: Reading received data from RX FIFO uses the Direct Read command (see Section 2.3.2) rather than the standard read sequence

- Modulation-Specific Status:
 - Status of LoRa® packets (`GetLoraPacketStatus(...)`),
 - Status of FSK packets (`GetFskPacketStatus(...)`),
 - and similar commands for other modulation types
- Statistics:
 - Reception statistics for LoRa® (`GetLoraRxStats(...)`),
 - Reception statistics for FSK (`GetFskRxstats(...)`),
 - and similar commands for other modulation types
- Special Features:
 - Results from ranging operations (`GetRangingResult(...)`),
 - Clear Channel Assessment results (`GetCcaResult(...)`),
 - a random number generated by the device (`GetRandomNumber(...)`)
- Register and Memory Access: Read from internal registers or memory (`ReadRegMem32(...)`)

2.3 FIFO Access

2.3.1 Direct Write Command

In the LR2021, direct write commands allow data to be written directly to the transmit FIFO buffer without requiring command processing at the NSS rising edge. This optimized mechanism enables high-speed data transfer for transmission by writing data byte-by-byte into the radio FIFO immediately after the opcode is parsed, significantly improving SPI operation efficiency.

2.3.2 Direct Read Command

The LR2021 has a special "direct read" command (`ReadRadioRxFifo(...)`) that operates differently from the standard read pattern. As described in Section 5.4.1.3 of the Datasheet, this command returns data in the same SPI frame as the command, without requiring a second SPI transaction. This speeds up data retrieval from the RX FIFO, making it more efficient to read received packets.

2.4 Monitoring FIFO Level

The LR2021 has two 256-byte data buffers (FIFOs) - one for transmission (TX FIFO) and one for reception (RX FIFO). Monitoring the fill levels of these FIFOs is crucial for efficient data management, especially when working with large data packets. The LR2021 offers three main methods for monitoring FIFO levels, each with its own advantages.

2.4.1 IRQ-Based FIFO Monitoring

The LR2021 can generate interrupts based on the FIFO fill levels, which allows an efficient and event-driven approach, reducing CPU overhead by only processing when needed. This is configured using the `ConfigFifoIrq(...)` command.

The available FIFO Flags are as follows:

- 0x01: FifoEmpty – FIFO is empty
- 0x02: FifoLow - FIFO level is below the low threshold
- 0x04: FifoHigh - FIFO level exceeds the high threshold
- 0x08: FifoFull - FIFO is full
- 0x10: FifoOverflow - FIFO has overflowed
- 0x20: FifoUnderflow - FIFO has underflowed

It is also necessary to map the IRQs to DIO Pins, calling the command `SetDioIrqConfig(DioPin, IRQ_RX_FIFO | IRQ_TX_FIFO)`. The host controller can monitor the DIO pin state, receiving an interrupt when the configured FIFO condition occurs. The host can then check which specific FIFO flag triggered the interrupt using command `GetFifoIrqFlags(...)`

For example, to receive large packets (>256 bytes), configure RX FIFO to trigger an IRQ when it is nearly full (e.g., at 200 bytes). When the IRQ triggers, read from the RX FIFO while continuing to receive data, preventing overflow.

2.4.2 Status Flag-Based FIFO Monitoring

Beyond IRQs, the LR2021 can directly map FIFO status flags to DIO pins, providing real-time status indication, using the command `SetDioFunction(DioPin, DIO_FUNCTION_FIFO_STATUS)`.

The specific status flag (empty, full, level threshold) is then mapped to the DIO pin. The DIO pin directly reflects the status of the selected FIFO condition, changing state immediately when the condition changes.

For example, map a "TX FIFO empty" flag to a DIO pin. After detecting the pin has gone high (indicating the FIFO is empty), the host knows it can immediately write a new batch of data without having to poll the device.

2.4.3 API-Based FIFO Level Monitoring

For more detailed control or when precise FIFO level information is needed, the LR2021 provides API commands to directly read the current FIFO levels.

- `GetRxFifoLevel(...)` returns the exact number of bytes currently in the RX FIFO. Similarly,
- `GetTxFifoLevel(...)` returns the exact number of bytes currently in the TX FIFO.

The host MCU can periodically poll these commands to get precise FIFO fill levels.

When implementing a flow control mechanism, periodically check `GetTxFifoLevel(...)` to determine how much more data can be written before filling the TX FIFO.

2.4.4 Practical Implementation Strategy

For optimal FIFO management, especially for large data transfers, a combined approach often works best:

For TX operations:

1. Configure TX FIFO low threshold IRQ to trigger when the FIFO has space for new data.
2. When the IRQ triggers, check `GetTxFifoLevel(...)` to determine exactly how much space is available.
3. Write new data to the FIFO using `WriteRadioTxFifo(...)`.

For RX Operations:

1. Configure RX FIFO high threshold IRQ to trigger when the FIFO is nearly full.
2. When the IRQ triggers, check `GetRxFifoLevel(...)` to determine how many bytes to read.
3. Read data from the FIFO using `ReadRadioRxFifo(...)`.
4. For continuous reception, clear the RX IRQ flag using `ClearFifoIrqFlags(...)`.

3 General Configuration Steps

For both transmission and reception, the following general steps apply regardless of the modulation used:

1. Configure general parameters: Set RF frequency, PA configuration, and DIO functions.
2. Select the specific packet type: Use `SetPacketType(...)` to define the modulation to be used.
3. Configure the modulation-specific parameters: Set modulation and packet parameters for the selected packet type.
4. Trigger the transmission or the reception:
 - For transmission: Load data to transmit FIFO and initiate TX.
 - For reception: Configure reception parameters and start RX.

Note that modulation-specific commands are available for packet type and modulation parameters of the supported modulations.

The following steps are similar across all modulation types:

1. Set standby mode as default state to start the transmission or reception of the radio packets.
2. Select RF frequency.
3. Configure PA (Power Amplifier).
4. Set TX power and ramp time.
5. Write data to TX FIFO and start transmission, or start reception and then read received data from RX FIFO.

Note: `SetPA (...)` command allows to switch between PAs, without wanting to re-configure the parameters of the PAs. It has then to be called after the `SetPaConfig (...)` command

4 LoRa® Packet Transmission and Reception

The LR2021 provides support for LoRa® modulation in both sub-GHz and 2.4 GHz bands. The following section describes the necessary steps to transmit or receive a LoRa packet for the LR2021.

Refer to <https://github.com/Lora-net/usp> and https://github.com/Lora-net/usp_zephyr for implementation examples.

4.1 LoRa Specific Configurations

The following parameters are specific for LoRa modulation. All the related commands shall be called for a successful LoRa packet transmission or reception:

- Packet Type Selection: `SetPacketType(0x00)` for LoRa®
- Modulation Parameters: `SetLoraModulationParams(SF, BW, CR, LDRO)`
 - Spreading Factor (SF) configuration
 - Bandwidth (BW) setting
 - Coding Rate (CR) selection
 - Low Data Rate Optimization (LDRO)
- Packet Configuration: `SetLoraPacketParam(preamble_length, header_type, payload_length, CRC, IQ_inversion)`
- Syncword Setting: `SetLoraSyncword(syncword)` for network identification

The LoRa modem offers Multi-SF Detection (optional) configurable with the command `SetLoraSideDetConfig()` and `SetLoraSideDetSyncword()`.

The Channel Activity Detection (CAD) feature requires also a dedicated configuration, using the command `SetLoraCadParams()`.

These features are described in AN1200.102 LR2021 LoRa Performance and in the LR2021 Datasheet.

4.2 Complete Flow for a LoRa Packet Transmission

The complete sequence of operations to trigger a LoRa® Packet Transmission is as follows:

1. Set Standby mode: for example, `SetStandby(0x01) // StandbyXOSC` mode.
2. Set packet type to LoRa®: `SetPacketType(0x00) // LoRa® (default)`.
3. Configure RF frequency: `SetRfFrequency(frequency_in_Hz) // e.g., 915000000 for 915MHz`.
4. Configure PA settings: `SetPaConfig(0x01, 0x00, 0x06, 0x07, 0x00) // Parameters depend on frequency band`.
5. Set TX power and ramp time: `SetTxParams(power, ramp_time) // Power range: -9.5 to 22dBm for sub-GHz`.
6. Configure LoRa® modulation parameters: `SetLoraModulationParams(SF, BW, CR, LDRO)`, where:
 - SF: Spreading Factor (5-12)
 - BW: Bandwidth (typically 0x04 for 125kHz)
 - CR: Coding Rate (0x01-0x08)
 - LDRO: Low Data Rate Optimization (0-1)
7. Configure LoRa® packet parameters: `SetLoraPacketParam(preamble_length, header_type, payload_length, CRC, IQ_inversion)`, where:
 - preamble_length: Typically 8-12 symbols
 - header_type: 0 for explicit, 1 for implicit
 - payload_length: Length in bytes

- CRC: 0 for disabled, 1 for enabled
- IQ_inversion: 0 for standard, 1 for inverted.

The device firmware automatically configures the IQ inversion depending on the configured BW.

8. Write data to transmit FIFO: `WriteRadioTxFifo(data_array, data_length)`.
9. Start transmission: `SetTx(timeout)` // timeout in 32kHz clock periods, 0 for no timeout.
10. Wait for TX completion by monitoring IRQ status.

4.3 LoRa® Packet Reception

The following steps are necessary to allow a LoRa® Packet reception:

1. Set Standby mode: for example `SetStandby(0x01)` // StandbyXOSC mode.
2. Set packet type to LoRa®: `SetPacketType(0x00)` // LoRa® (default).
3. Configure RF frequency: `SetRfFrequency(frequency_in_Hz)` // e.g., 915000000 for 915MHz.
4. Configure RX path: `SetRxPath(0x00, 0x00)` // RxPath=0 for LF, RxBoost=0 for normal mode.
5. Configure LoRa® modulation parameters: `SetLoraModulationParams(SF, BW, CR, LDRO)`. Ensure that these parameters match with the transmitter.
6. Configure LoRa® packet parameters: `SetLoraPacketParam(preamble_length, header_type, payload_length, CRC, IQ_inversion)`. Match these parameters with the transmitter.

Note: when `payload_length` is set to 0: the device accepts any payload length (no length filtering). When set to 1-255: the device only accepts payloads with lengths from 1 to the specified value. Payloads with length 0 or exceeding this value will be rejected, triggering a header error interrupt (`IRQ_HEADER_ERROR`).

7. Configure IRQ settings for reception: `SetDioIrqConfig(dio_pin, IRQ_RX_DONE | IRQ_CRC_ERROR | IRQ_TIMEOUT)`.
8. Start reception: `SetRx(timeout)` // timeout in 32kHz clock periods, 0xFFFF for continuous mode.
9. Wait for reception by monitoring `IRQ_RX_DONE`.
10. Read received data: `GetRxPktLength(...)` // Get received packet length.
11. `ReadRadioRxFifo(buffer, length)` // Read data from RX FIFO.
12. Check packet status: `GetLoraPacketStatus(...)` // Get RSSI, SNR, and error information.

You can also configure multi-SF detection (to allow detection of multiple spreading factors simultaneously) using commands:

- `SetLoraSideDetConfig(Config1, Config2, Config3)` and
- `SetLoraSideDetSyncword(Syncword1, Syncword2, Syncword3)`.

5 FSK Packet Transmission and Reception

FSK (Frequency-Shift Keying) is a versatile modulation scheme that achieves high data rates. The LR2021 supports FSK, GFSK, MSK, and GMSK variants.

Refer to <https://github.com/Lora-net/usp> and https://github.com/Lora-net/usp_zephyr for implementation examples.

5.1 FSK Specific Configurations

The following parameters are specific for FSK. It is necessary to configure them using the commands dedicated to FSK modulation:

- Packet Type Selection: `SetPacketType(0x02)` for FSK
- Modulation Parameters: `SetFskModulationParams(bit_rate, pulse_shape, rx_bandwidth, frequency_deviation)`
- Packet Configuration: `SetFskPacketParams(preamble_length, preamble_detection, address_comp, packet_format, payload_length, CRC_type, whitening)`
- Whitening Configuration (optional): `SetFskWhiteningParams(whitening_type, initial_value)`
- CRC Configuration (optional): `SetFskCrcParams(polynomial, initial_value)`
- Syncword Setting: `SetFskSyncword(syncword, bit_order, num_bits)`
- Address Configuration (optional): `SetFskAddress(node_address, broadcast_address)`

5.2 FSK Packet Transmission

The sequence of operations to trigger an FSK Packet Transmission is as follows:

1. Set Standby mode: for example `SetStandby(0x01)` // StandbyXOSC mode.
2. Set packet type to FSK: `SetPacketType(0x02)` // FSK.
3. Configure RF frequency: `SetRfFrequency(frequency_in_Hz)` // e.g., 868000000 for 868MHz.
4. Configure PA settings: `SetPaConfig(0x01, 0x00, 0x06, 0x07, 0x00)` // Parameters depend on frequency band.
5. Set TX power and ramp time: `SetTxParams(power, ramp_time)` // Power range: -9.5 to 22dBm for sub-GHz.
6. Configure FSK modulation parameters: `SetFskModulationParams(bit_rate, pulse_shape, rx_bandwidth, frequency_deviation)`, where:
 - `bit_rate`: Bit rate in bps (e.g., 50000 for 50kbps)
 - `pulse_shape`: 0x00 for no shaping, 0x04 for Gaussian BT=0.3
 - `rx_bandwidth`: Receiver bandwidth value
 - `frequency_deviation`: Frequency deviation in Hz
7. Configure FSK packet parameters: `SetFskPacketParams(preamble_length, preamble_detection, address_comp, packet_format, payload_length, CRC_type, whitening)`, where:
 - `preamble_length`: Number of preamble bits
 - `preamble_detection`: 0x00 for off, other values for bit length
 - `address_comp`: 0x00 for disabled, 0x01 for node address, 0x02 for node and broadcast
 - `packet_format`: 0x00 for fixed length, 0x01 for variable length
 - `payload_length`: Fixed length in bytes or maximum variable length
 - `CRC_type`: 0x00 for no CRC, 0x01-0x04 for 1-4 byte CRC
 - `whitening`: 0x00 for disabled, 0x01 for enabled

8. Configure whitening if enabled: `SetFskWhiteningParams(whitening_type, initial_value)`.
9. Configure CRC if enabled: `SetFskCrcParams(polynomial, initial_value)`.
10. Set Syncword: `SetFskSyncword(syncword, bit_order, num_bits)`.
11. Set address if address filtering is enabled: `SetFskAddress(node_address, broadcast_address)`.
12. Write data to transmit FIFO: `WriteRadioTxFifo(data_array, data_length)`.
13. Start transmission: `SetTx(timeout)` // timeout in 32kHz clock periods, 0 for no timeout.
14. Wait for TX completion by monitoring IRQ status.

5.3 FSK Packet Reception

The sequence of operations to trigger an FSK® Packet reception is as follows:

1. Set Standby mode: for example `SetStandby(0x01)` // StandbyXOSC mode.
2. Set packet type to FSK: `SetPacketType(0x02)` // FSK.
3. Configure RF frequency: `SetRfFrequency(frequency_in_Hz)` // e.g., 868000000 for 868MHz.
4. Configure RX path: `SetRxPath(0x00, 0x00)` // RxPath=0 for LF, RxBoost=0 for normal mode.
5. Configure FSK modulation parameters: `SetFskModulationParams(bit_rate, pulse_shape, rx_bandwidth, frequency_deviation)`. These parameters should be the same as for packet transmission.
6. Configure FSK packet parameters: `SetFskPacketParams(preamble_length, preamble_detection, address_comp, packet_format, payload_length, CRC_type, whitening)`. Match these parameters with the transmitter.
7. Configure whitening if enabled: `SetFskWhiteningParams(whitening_type, initial_value)`.
8. Configure CRC if enabled: `SetFskCrcParams(polynomial, initial_value)`.
9. Set Syncword: `SetFskSyncword(syncword, bit_order, num_bits)`.
10. Set address if address filtering is enabled: `SetFskAddress(node_address, broadcast_address)`.
11. Configure IRQ settings for reception: `SetDioIrqConfig(dio_pin, IRQ_RX_DONE | IRQ_CRC_ERROR | IRQ_PREAMBLE_DETECTED | IRQ_SYNCWORD_VALID)`.
12. Start reception: `SetRx(timeout)` // timeout in 32kHz clock periods, 0xFFFF for continuous mode.
13. Wait for reception by monitoring `IRQ_RX_DONE`.
14. Read received data:
 1. `GetRxPktLength(...)` // Get received packet length.
 2. `ReadRadioRxFifo(buffer, length)` // Read data from RX FIFO.
15. Check packet status: `GetFskPacketStatus(...)` // Get RSSI and status information.

6 Sending/Receiving Long GFSK Packets

This section describes the necessary operations to transmit GFSK payloads larger than 256 bytes.

The transceiver can send payloads larger than the internal FIFO. Since this FIFO is only 256 bytes, it will need to be written to multiple times while transmitting to send long payloads.

Every time a byte is transmitted, it is removed from the FIFO. Sending large payloads requires that the TX FIFO is refilled whenever it gets under a certain threshold of data.

When receiving large payloads, you need to keep reading from the FIFO when it is filled above a certain threshold.

The FIFO is accessible while receiving and transmitting.

The following flowchart describes the necessary steps for transmitting payloads larger than 256 bytes.

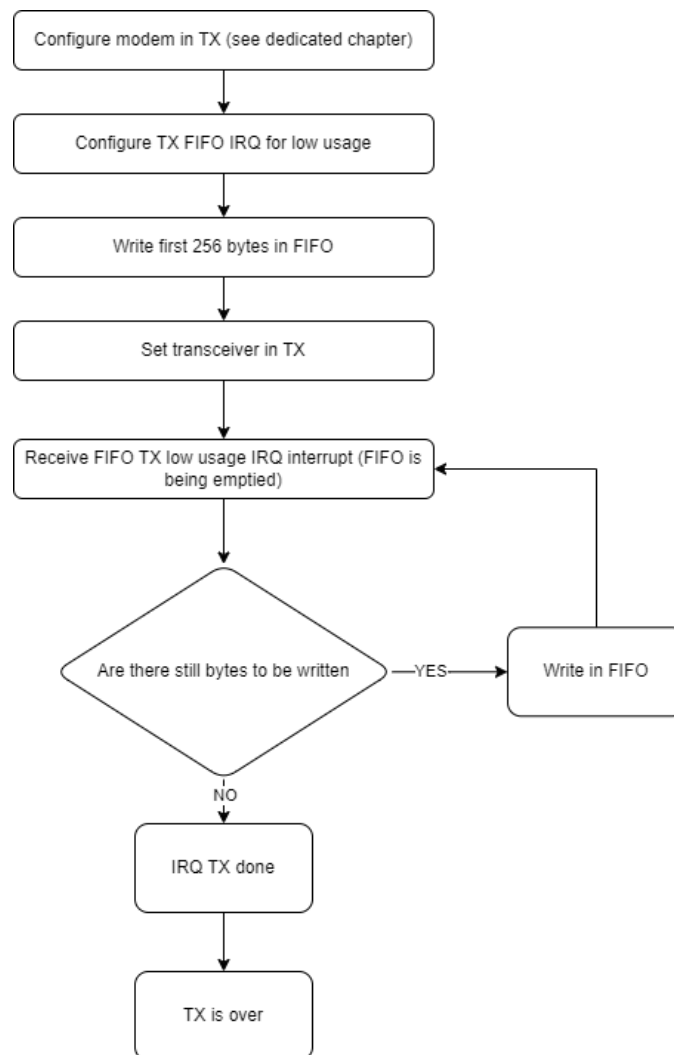


Figure 1: Flowchart for transmitting payloads larger than 256 bytes.

This following flowchart shows the concepts behind receiving payloads larger than 256 bytes.

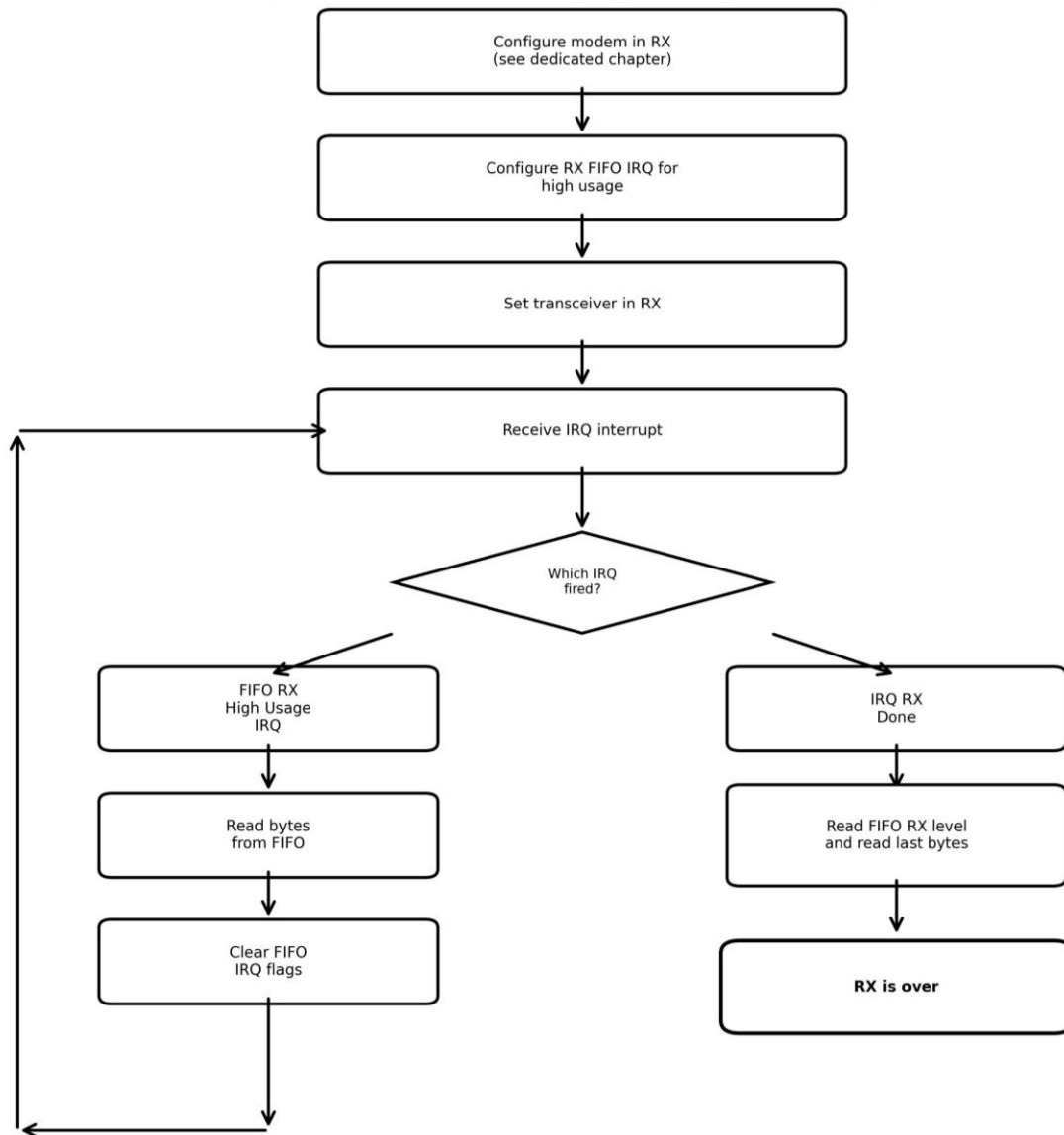


Figure 2: Flowchart for receiving payload larger than 256 bytes

7 FLRC Packet Transmission and Reception

FLRC (Fast Long-Range Communication) is a high-speed modulation scheme that offers improved sensitivity compared to FSK at the same data rate. This section describes the steps to send or receive an FLRC radio packet.

Refer to <https://github.com/Lora-net/usp> and https://github.com/Lora-net/usp_zephyr for implementation examples.

7.1 FLRC Specific Configurations

The following parameters are specific for FLRC modulation. All the related commands shall be called for a successful FLRC packet transmission or reception:

- Packet Type Selection: `SetPacketType(0x05)` for FLRC
- Modulation Parameters: `SetFlrcModulationParams(bit_rate_bandwidth, coding_rate, pulse_shape)`
- Packet Configuration: `SetFlrcPacketParams(agc_preamble_length, sync_word_length, sync_word_tx, sync_word_match, packet_format, CRC_type, payload_length)`
- Syncword Setting: `SetFlrcSyncword(syncword_number, syncword_value)`

7.2 FLRC Packet Transmission

The sequence of operations to trigger an FLRC® Packet Transmission is as follows:

1. Set Standby mode: for example `SetStandby(0x01)` // StandbyXOSC mode.
2. Set packet type to FLRC: `SetPacketType(0x05)` // FLRC.
3. Configure RF frequency: `SetRfFrequency(frequency_in_Hz)` // e.g., 2450000000 for 2.45 GHz.
4. Configure PA settings: `SetPaConfig(0x00, 0x00, 0x00, 0x00, 0x10)` // Parameters for 2.4 GHz.
5. Set TX power and ramp time: `SetTxParams(power, ramp_time)` // Power range: -19.5 to 12 dBm for 2.4 GHz.
6. Configure FLRC modulation parameters: `SetFlrcModulationParams(bit_rate_bandwidth, coding_rate, pulse_shape)`, where:
 - `bit_rate_bandwidth`: 0x00-0x07 for different bit rate and bandwidth combinations
 - `coding_rate`: 0x00 for CR 1/2, 0x01 for CR 3/4, 0x02 for CR 1, 0x03 for CR 2/3
 - `pulse_shape`: Same values as FSK
7. Configure FLRC packet parameters: `SetFlrcPacketParams(agc_preamble_length, sync_word_length, sync_word_tx, sync_word_match, packet_format, CRC_type, payload_length)`, where:
 - `agc_preamble_length`: 0x00-0x07 for 4-32 bits
 - `sync_word_length`: Length in 16-bit units
 - `sync_word_tx`: Syncword selection for TX
 - `sync_word_match`: Syncword matching mode for RX
 - `packet_format`: 0 for variable length, 1 for fixed length
 - `CRC_type`: 0 for off, 1-3 for 16/24/32 bit CRC
 - `payload_length`: Length in bytes (6-511)
8. Configure Syncword: `SetFlrcSyncword(syncword_number, syncword_value)`.
9. Write data to transmit FIFO: `WriteRadioTxFifo(data_array, data_length)`.

10. Start transmission: `SetTx(timeout)` // timeout in 32kHz clock periods, 0 for no timeout.
11. Wait for TX completion by monitoring IRQ status.

7.3 Required Commands Sequence for FLRC Reception

The following steps are necessary to allow an FLRC® Packet reception:

1. Set Standby mode: for example `SetStandby(0x01)` // StandbyXOSC mode.
2. Set packet type to FLRC: `SetPacketType(0x05)` // FLRC.
3. Configure RF frequency: `SetRfFrequency(frequency_in_Hz)` // e.g., 2450000000 for 2.45 GHz.
4. Configure RX path: `SetRxPath(0x01, 0x00)` // RxPath=1 for HF, RxBoost=0 for normal mode.
5. Configure FLRC modulation parameters: `SetFlrcModulationParams(bit_rate_bandwidth, coding_rate, pulse_shape)`. Match these parameters with the transmitter.
6. Configure FLRC packet parameters: `SetFlrcPacketParams(agc_preamble_length, sync_word_length, sync_word_tx, sync_word_match, packet_format, CRC_type, payload_length)`. Match these parameters with the transmitter.
7. Configure Syncword: `SetFlrcSyncword(syncword_number, syncword_value)`.
8. Configure IRQ settings for reception: `SetDioIrqConfig(dio_pin, IRQ_RX_DONE | IRQ_CRC_ERROR | IRQ_SYNCWORD_VALID)`.
9. Start reception: `SetRx(timeout)` // timeout in 32kHz clock periods, 0xFFFF for continuous mode.
10. Wait for reception by monitoring `IRQ_RX_DONE`.
11. Read received data:
 1. `GetRxPktLength(...)` // Get received packet length.
 2. `ReadRadioRxFifo(buffer, length)` // Read data from RX FIFO.
12. Check packet status: `GetFlrcPacketStatus(...)` // Get RSSI and status information.

8 OOK Packet Transmission and Reception

The LR2021 supports OOK (On-Off Keying) modulation, for which the carrier is either on or off to represent binary data. The following section describes the necessary steps to transmit or receive an OOK packet with the LR2021.

8.1 OOK Specific Configurations

The following parameters are specific for OOK modulation. All the related commands shall be called for a successful OOK packet transmission or reception:

- Packet Type Selection: `SetPacketType(0x0A)` for OOK
- Modulation Parameters: `SetOokModulationParams(bit_rate, pulse_shape, rx_bandwidth, depth)`, where Depth parameter is unique to OOK
- Packet Configuration: `SetOokPacketsParams(preamble_length, address_comp, packet_format, payload_length, CRC_type, manchester)`, where Manchester encoding is specific to OOK
- CRC Configuration: `SetOokCrcParams(polynomial, initial_value)`
- Syncword Setting: `SetOokSyncword(syncword, bit_order, num_bits)`
- Address Configuration: `SetOokAddress(node_address, broadcast_address)`
- Detector Configuration: `SetOokDetector(preamble_pattern, pattern_length, pattern_num_repeats, sw_is_raw, sfd_kind, sfd_length)`
- Whitening Configuration: `SetOokwhiteningParams(bit_idx, polynomial, init_value)`

8.2 OOK Packet Transmission

The following steps are necessary to allow OOK Packet transmission:

1. Set Standby mode: for example `SetStandby(0x01) // StandbyXOSC. mode`
2. Set packet type to OOK: `SetPacketType(0x0A) // OOK.`
3. Configure RF frequency: `SetRfFrequency(frequency_in_Hz) // e.g., 433000000 for 433MHz.`
4. Configure PA settings: `SetPaConfig(0x01, 0x00, 0x06, 0x07, 0x00) // Parameters depend on frequency band.`
5. Set TX power and ramp time: `SetTxParams(power, ramp_time) // Power range: -9.5 to 22dBm for sub-GHz.`
6. Configure OOK modulation parameters: `SetOokModulationParams(bit_rate, pulse_shape, rx_bandwidth, depth)`, where:
 - bit_rate: Bit rate in bps (e.g., 4800 for 4.8 kbps)
 - pulse_shape: 0x00 for no shaping, other values as for FSK
 - rx_bandwidth: Receiver bandwidth value
 - depth: Optional, 0 for full depth, 1 for up to 20dB
7. Configure OOK packet parameters: `SetOokPacketsParams(preamble_length, address_comp, packet_format, payload_length, CRC_type, manchester)`, where:
 - preamble_length: Number of preamble bits
 - address_comp: 0x00 for disabled, 0x01 for node address, 0x02 for node and broadcast
 - packet_format: 0x00 for fixed length, 0x01 for variable length
 - payload_length: Fixed length in bytes or maximum variable length
 - CRC_type: 0x00 for no CRC, 0x01-0x04 for 1-4 byte CRC

- manchester: 0 for disabled, 1 for enabled, 3 for inverted
- 8. Configure CRC if enabled: `SetOokCrcParams(polynomial, initial_value)`.
- 9. Set Syncword: `SetOokSyncword(syncword, bit_order, num_bits)`.
- 10. Set address if address filtering is enabled: `SetOokAddress(node_address, broadcast_address)`.
- 11. Configure whitening if needed: `SetOokwhiteningParams(bit_idx, polynomial, init_value)`.
- 12. Write data to transmit FIFO: `WriteRadioTxFifo(data_array, data_length)`.
- 13. Start transmission: `SetTx(timeout)` // timeout in 32kHz clock periods, 0 for no timeout.
- 14. Wait for TX completion by monitoring IRQ status.

8.3 OOK Packet Reception

The following steps are necessary to allow OOK® Packet reception:

1. Set Standby mode: for example `SetStandby(0x01)` // StandbyXOSC mode.
2. Set packet type to OOK: `SetPacketType(0x0A)` // OOK.
3. Configure RF frequency: `SetRfFrequency(frequency_in_Hz)` // e.g., 433000000 for 433MHz.
4. Configure RX path: `SetRxPath(0x00, 0x00)` // RxPath=0 for LF, RxBoost=0 for normal mode.
5. Configure OOK modulation parameters: `SetOokModulationParams(bit_rate, pulse_shape, rx_bandwidth, depth)`. These parameters should match with the transmitter.
6. Configure OOK packet parameters: `SetOokPacketsParams(preamble_length, address_comp, packet_format, payload_length, CRC_type, manchester)`. These parameters should match with the transmitter.
7. Configure CRC if enabled: `SetOokCrcParams(polynomial, initial_value)`.
8. Set Syncword: `SetOokSyncword(syncword, bit_order, num_bits)`.
9. Set address if address filtering is enabled: `SetOokAddress(node_address, broadcast_address)`.
10. Configure whitening if needed: `SetOokwhiteningParams(bit_idx, polynomial, init_value)`.
11. Configure OOK detector: `SetOokDetector(preamble_pattern, pattern_length, pattern_num_repeats, sw_is_raw, sfd_kind, sfd_length)`.
12. Configure IRQ settings for reception: `SetDioIrqConfig(dio_pin, IRQ_RX_DONE | IRQ_CRC_ERROR | IRQ_SYNCWORD_VALID)`.
13. Start reception: `SetRx(timeout)` // timeout in 32kHz clock periods, 0xFFFF for continuous mode.
14. Wait for reception by monitoring `IRQ_RX_DONE`.
15. Read received data: `GetRxPktLength(...)` // Get received packet length, and `ReadRadioRxFifo(buffer, length)` // Read data from RX FIFO.
16. Check packet status: `GetOokPacketstatus(...)` // Get RSSI and status information.

9 BPSK Packet Transmission

The LR2021 supports BPSK (Binary Phase-Shift Keying) and DBPSK (Differential Binary Phase-Shift Keying) modulation for transmit-only operations for applications requiring very long range at low data rates, such as IoT sensor networks, telemetry systems, and other low-power wide-area network (LPWAN) applications, like Sigfox. It supports both 100bps and 600bps data rates.

9.1 BPSK Specific Configurations

The following commands are specific for BPSK. It is necessary to use these commands to configure the BPSK parameters:

- `SetBpskModulationParams (0x0250)` : Configure modulation parameters
- `SetBpskPacketParams (0x0251)` : Configure packet parameters

9.2 Sequence for BPSK Packet Transmission

The Required Command Sequence for BPSK Transmission is as follows:

1. Set Standby mode: for example `SetStandby(0x01)` // StandbyXOSC mode.
2. Set packet type to BPSK: `SetPacketType(0x06)` // BPSK.
3. Configure RF frequency: `SetRfFrequency(frequency_in_Hz)` // e.g., 868000000 for 868MHz.
4. Configure PA settings: `SetPaConfig(0x01, 0x00, 0x06, 0x07, 0x00)` // Parameters depend on frequency band.
5. Set TX power and ramp time: `SetTxParams(power, ramp_time)` // Power range: -9.5 to 22dBm for sub-GHz.
6. Configure BPSK modulation parameters: `SetBpskModulationParams(bit_rate, pulse_shape, diff_mode, diff_mode_init)`, where:
 - `bit_rate`: Bit rate in bps (up to 600bps)
 - When `BitRate(31) = 0`: BitRate is taken directly as `BitRate(30:0)` in bits per second
 - When `BitRate(31) = 1`: BitRate is calculated as `BitRate(30:0)/256` for fractional values
 - `pulse_shape`: Symbol pulse shaping filter (0x0-0xF):
 - 0x0: No pulse shaping
 - 0x1: Use custom coefficients
 - 0x2: Gaussian with BT 2.0
 - 0x3: Root-Raised-Cosine with Roll-Off 0.4
 - 0x4: Gaussian with BT 0.3
 - 0x5: Gaussian with BT 0.5
 - 0x6: Gaussian with BT 0.7
 - 0x7: Gaussian with BT 1.0
 - 0x8: Raised-Cosine with Roll-Off 0.3
 - 0x9: Raised-Cosine with Roll-Off 0.5
 - 0xA: Raised-Cosine with Roll-Off 0.7
 - 0xB: Raised-Cosine with Roll-Off 1.0
 - 0xC: Root-Raised-Cosine with Roll-Off 0.3

- 0xD: Root-Raised-Cosine with Roll-Off 0.5
- 0xE: Root-Raised-Cosine with Roll-Off 0.7
- 0xF: Root-Raised-Cosine with Roll-Off 1.0
- diff_mode:
 - 1: Enable differential encoding (DBPSK)
 - 0: Disable differential encoding (BPSK)
- diff_mode_init: Initial value for differential encoder (2 bits)
 - Bit 0: Previous value
 - Bit 1: Parity
 - Discarded if diff_mode = 0

For example, for Sigfox, we would have the following configuration:

- bit_rate: 100 or 600bps (typical Sigfox uplink rates)
 - pulse_shape: 0x4 (Gaussian with BT 0.3) – (to comply with spectral requirements)
 - diff_mode: 1 (Enable differential encoding - DBPSK)
 - diff_mode_init: 0x0 (Initial value for differential encoder)
7. Configure BPSK packet parameters: `SetBpskPacketParams(payload_length, mode, bpsk_s_msg, bpsk_s_rank)`, where:
- payload_length:
 - Payload length in bits when Mode = 0 (raw encoded)
 - Payload length in bytes when Mode = 1 (BPSK_S PHY)
 - mode: Data handling mode
 - 0x0: Raw encoded - nothing added to FIFO data
 - 0x1: BPSK_S PHY - data corresponds to UL-CONTAINER with automatic CRC, convolution, frame type, and preamble handling
 - bpsk_s_msg: Sigfox message type (when mode = 1)
 - 0: Application message
 - 1: Control message
 - bpsk_s_rank: Sigfox frame emission rank (when mode = 1)
 - 0x0: First frame or single frame
 - 0x1: Second frame
 - 0x2: Third frame

For example, for Sigfox, we would have the following configuration:

- payload_length: Depends on your Sigfox message size (in bytes when mode = 1). Typically 12 bytes maximum for Sigfox uplink messages
 - mode: 0x1 (BPSK_S PHY mode) - This is crucial for Sigfox. This mode automatically handles CRC, convolution coding, frame type, and preamble according to Sigfox specifications
 - bpsk_s_msg: Message type and bpsk_s_rank: as per the Sigfox application requirements.
8. Start transmission: `SetTx(timeout)` // timeout in 32kHz clock periods, 0 for no timeout.
9. Wait for TX completion by monitoring IRQ status.

10 LR-FHSS Packet Transmission

LR-FHSS (Long Range Frequency Hopping Spread Spectrum) is a transmit-only modulation technique designed for long-range communication with multiple devices occupying the same spectrum.

10.1 LR-FHSS Specific Configurations

The following parameters are specific for LR-FHSS. It is necessary to configure them using the commands dedicated to LR-FHSS modulation:

- Packet type selection: `SetPacketType(0x07) // LR-FHSS`
- Syncword setting: `LrFhssSetSyncword(syncword) // Default is 0x2C0F7995`
- LR-FHSS frame build: `LrFhssBuildFrame(sync_header_count, coding_rate, mod_type, grid, hopping, bandwidth, sequence, offset, payload)`

10.2 LR-FHSS Packet Transmission

The Required Command Sequence for LR-FHSS Transmission are as follows:

10. Set Standby mode: for example `SetStandby(0x01) // StandbyXOSC mode`.
11. Set packet type to LR-FHSS: `SetPacketType(0x07) // LR-FHSS`.
12. Configure RF frequency: `SetRfFrequency(frequency_in_Hz) // e.g., 868000000 for 868MHz`.
13. Configure PA settings: `SetPaConfig(0x01, 0x00, 0x06, 0x07, 0x00) // Parameters depend on frequency band`.
14. Set TX power and ramp time: `SetTxParams(power, ramp_time) // Power range: -9.5 to 22dBm for sub-GHz`.
15. Configure LR-FHSS Syncword (optional, has default value): `LrFhssSetSyncword(syncword) // Default is 0x2C0F7995`.
16. Build LR-FHSS frame: `LrFhssBuildFrame(sync_header_count, coding_rate, mod_type, grid, hopping, bandwidth, sequence, offset, payload)`, where:
 - `sync_header_count`: 1-4 sync headers
 - `coding_rate`: 0x00 for CR 5/6, 0x01 for CR 2/3, 0x02 for CR 1/2, 0x03 for CR 1/3
 - `mod_type`: 0 for GMSK at 488.28125Hz
 - `grid`: 0 for 25.390625kHz, 1 for 3.90625kHz
 - `hopping`: 0 for no hopping, 1 for hopping, 2/3 for test modes
 - `bandwidth`: 0-9 according to LR-FHSS specifications
 - `sequence`: 16-bit hopping sequence seed
 - `offset`: Frequency offset within valid range
 - `payload`: Data bytes to transmit
17. Start transmission: `SetTx(timeout) // timeout in 32kHz clock periods, 0 for no timeout`.
18. Wait for TX completion by monitoring IRQ status.

Note: LR-FHSS is a transmit-only mode in the LR2021. The reception is ensured by Gateway V2 designs.

11 DIO RX/TX Trigger

Packet transmission and reception can be triggered using DIO pins, in addition to SPI commands.

Using DIO pins to trigger TX/RX operations offers significant advantages over SPI command-based triggering:

- **Reduced Latency:** Eliminates SPI bus transaction time and internal command parsing overhead, enabling faster response to triggering events.
- **Precise Timing Control:** Allows the host MCU to trigger transmissions or receptions at precise moments using hardware timers or other GPIO-driven mechanisms. This is particularly valuable for time-synchronized networks, scheduled transmissions, or applications requiring deterministic timing that cannot be achieved through software-based SPI command sequences.
- **Lower Host CPU Overhead:** The host can configure the trigger once and use simple GPIO toggling rather than constructing and transmitting full SPI command sequences.

The necessary steps are the following:

- Configure the DIO(s) to be used as TX and/or RX triggers
- Set the default RX/TX timeout to be used when triggering RX or TX
- Set the configured DIOs as logical HIGH to trigger RX or TX
- Reset the pin to LOW once the operation is completed, or after the minimum HIGH time

The following example shows how to configure a DIO to act as a TX trigger (this example is based on the LR2021 C driver and uses DIO number 8):

```
lr20xx_system_dio_t      dio = LR20XX_SYSTEM_DIO_8;
lr20xx_system_dio_func_t dio_function = LR20XX_SYSTEM_DIO_FUNC_TX_TRIGGER;
lr20xx_system_dio_drive_t dio_drive = LR20XX_SYSTEM_DIO_DRIVE_NONE;
lr20xx_system_set_dio_function( context, dio, dio_function, dio_drive );
```

The following example shows how to configure a DIO to act as an RX trigger (this example is based on the LR2021 C driver and uses DIO number 8):

```
lr20xx_system_dio_t      dio = LR20XX_SYSTEM_DIO_8;
lr20xx_system_dio_func_t dio_function = LR20XX_SYSTEM_DIO_FUNC_RX_TRIGGER;
lr20xx_system_dio_drive_t dio_drive = LR20XX_SYSTEM_DIO_DRIVE_NONE;
lr20xx_system_set_dio_function( context, dio, dio_function, dio_drive );
```


12 Advanced APIs for Standard-based Protocols

The LR2021 offers advanced APIs, easing the setup of the Physical Layer characteristics of the transmitter and receiver, in the context of:

- Wi-SUN
- Bluetooth Low Energy
- WM-BUS
- Z-Wave

The LR2021 is a transceiver and must be used in conjunction with a host MCU running the upper layer protocol stack, but it is easily setup thanks to these settings.

Disclaimer: Semtech's products are designed to be used in connection with qualified Bluetooth® products and applications but are not certified or qualified Bluetooth® products.

12.1 OQPSK_15_4 Packets

OQPSK_15_4 supports the IEEE 802.15.4 physical layer for 2 Mega Chips per Second (Mcps) operation with a data rate of 250 kbps.

The OQPSK_15_4 API set is described in the LR2021 datasheet.

12.2 BLE Packets

The LR2021 supports the Bluetooth® Low Energy (BLE) physical layer up to version 5.1. The BLE API set is described in the LR2021 datasheet. Please contact your Semtech representative to review the stack offering.

12.3 WM-BUS Packets

WM-BUS (Wireless M-Bus) is a standard for wireless meter reading, whose physical layer is also supported by the LR2021.

The LR2021 supports the following WM-bus modes:

- Mode C1 (TX) and C2 (meter RX/TX) are optimized for use in Battery-Powered Smart Metering applications, offering power-efficient communication.
- Mode N is tailored for Narrowband applications, providing efficient data transmission in constrained bandwidth environments.
- Mode S1 is intended for Smart Metering applications, optimizing communication for advanced metering systems.
- Mode S2 mode builds upon S1, provides additional features and improvements for more sophisticated metering solutions.
- Mode T1 is designed for use in sub-GHz applications, offering reliable and robust data transfer in this frequency range.
- Mode T2 builds upon T1, introducing further enhancements to meet the demands of sub-GHz communication scenarios.
- Mode R2 is specific to radio-controlled devices, enabling efficient communication in remote-controlled applications.
- Mode F2 introduces additional capabilities for specific WM-BUS applications, providing enhanced functionalities and performance for specialized use cases.

The WM-BUS API set is described in the LR2021 datasheet. Please contact your Semtech representative to review the stack offering.

12.4 WI-SUN Packets

WI-SUN (Wireless Smart Utility Network) is a standard for wireless communication in utility networks. The LR2021 supports the 2-FSK physical layer (PHY) variant.

The WI-SUN API set is described in the LR2021 datasheet. Please contact your Semtech representative to review the stack offering.

12.5 Z-Wave Packets

Z-Wave is a wireless communication protocol for home automation devices.

The Z-Wave API set is described in the LR2021 datasheet.

13 Conclusion

This application note has provided detailed instructions for transmitting and receiving different packet types using the LR2021 multi-band LoRa® transceiver. By following the command sequences outlined for each modulation scheme, developers can effectively utilize the diverse capabilities of the LR2021 in their wireless communication applications.

The LR2021's flexibility and support for multiple modulation schemes make it an ideal choice for a wide range of applications, from long-range low-power communication using LoRa®, to high-speed data transfer using FLRC, and compatibility with standard protocols like BLE, WM-BUS, WI-SUN, Z-Wave, and IEEE 802.15.4.

When implementing these command sequences, remember to:

1. Always check the BUSY signal before issuing commands.
2. Configure appropriate IRQs for event notification.
3. Match modulation and packet parameters between transmitter and receiver.
4. Select the appropriate PA and RF path based on the frequency band.
5. Consider the timing requirements for mode switching operations detailed in the LR2021 datasheet.

By carefully following these guidelines, developers can harness the full potential of the LR2021 transceiver in their wireless applications.

14 Glossary

Term	Description
API	Application Interface
BPSK	Binary Phase Shift Keying
BW	Bandwidth
CAD	Channel Activity Detection
CR	Coding Rate
CRC	Cyclical Redundancy Check
DBPSK	Differential Binary Phase Shift Keying
DIO	Digital Input / Output
FIFO	First In First out
FLRC	Fast Long-Range Communication
(G)FSK	(Gaussian) Frequency-Shift Keying
(G) MSK	(Gaussian) Minimum Shift Keying
IRQ	Interrupt Request
LDRO	Low Data Rate Optimization
LoRa	Long Range Communication The LoRa mark is a registered trademark of the Semtech organisation
LR-FHSS	Long Range Frequency Hopping Spread Spectrum
MCPS	Mega Chips Per Second
MCU	Micro Controller Unit
MISO	Master Input Slave Output
MOSI	Master Output Slave Input

NSS	Slave Select active low
OOK	On-Off Keying
OQPSK	Offset Quadrature Phase Shift Keying
PA	Power Amplifier
PDU	Protocol Data Unit
RSSI	Received Signal Strength Indicator
RX	Reception
SF	Spreading Factor
SNR	Signal to Noise Ration
SPI	Serial Peripheral Interface
TX	Transmission

15 Revision History

Version	ECO	Date	Changes and/or Modifications
1.0	076673	October 2025	Initial Release



Important Notice

Information relating to this product and the application or design described herein is believed to be reliable, however such information is provided as a guide only and Semtech assumes no liability for any errors in this document, or for the application or design described herein.

Semtech reserves the right to make changes to the product or this document at any time without notice. Buyers should obtain the latest relevant information before placing orders and should verify that such information is current and complete. Semtech warrants performance of its products to the specifications applicable at the time of sale, and all sales are made in accordance with Semtech's standard terms and conditions of sale.

SEMTECH PRODUCTS ARE NOT DESIGNED, INTENDED, AUTHORIZED OR WARRANTED TO BE SUITABLE FOR USE IN LIFE-SUPPORT APPLICATIONS, DEVICES OR SYSTEMS, OR IN NUCLEAR APPLICATIONS IN WHICH THE FAILURE COULD BE REASONABLY EXPECTED TO RESULT IN PERSONAL INJURY, LOSS OF LIFE OR SEVERE PROPERTY OR ENVIRONMENTAL DAMAGE. INCLUSION OF SEMTECH PRODUCTS IN SUCH APPLICATIONS IS UNDERSTOOD TO BE UNDERTAKEN SOLELY AT THE CUSTOMER'S OWN RISK.

Should a customer purchase or use Semtech products for any such unauthorized application, the customer shall indemnify and hold Semtech and its officers, employees, subsidiaries, affiliates, and distributors harmless against all claims, costs damages and attorney fees which could arise.

The Semtech name and logo are registered trademarks of the Semtech Corporation. All other trademarks and trade names mentioned may be marks and names of Semtech or their respective companies. Semtech reserves the right to make changes to, or discontinue any products described in this document without further notice. Semtech makes no warranty, representation or guarantee, express or implied, regarding the suitability of its products for any particular purpose. All rights reserved.

© Semtech 2025